

```
1 import numpy as np
2 def AUC(y_scores,y_true):
3     # 按照预测分数升序排序
4     indexs=sorted(range(len(y_scores)),key=lambda i:y_scores[i])
5     # 按照排序后的索引提取真实标签
6     sorted_labels=[y_true[i] for i in indexs]
7     # 统计正负样本数量
8     pos_cnt=np.sum(y_true)
9     neg_cnt=len(y_true)-pos_cnt
10    # 计算所有正样本的秩次和
11    rank_sum=0
12    for i in range(len(sorted_labels)):
13        if sorted_labels[i]==1:
14            rank_sum += i
15    auc = (rank_sum-pos_cnt*(pos_cnt-1)/2)/(pos_cnt*neg_cnt)
16
17 def AUC_with_ties(y_scores, y_true):
18     # 创建(score, label, original_index)的元组
19     data = [(y_scores[i], y_true[i], i) for i in range(len(y_scores))]
20     # 按分数排序，分数相同时保持稳定
21     data.sort(key=lambda x: x[0])
22
23     # 处理相同分数的平均秩次
24     n = len(data)
25     ranks = [0] * n
26     i = 0
27     while i < n:
28         j = i
29         # 找到所有相同分数的样本
30         while j < n and data[j][0] == data[i][0]:
31             j += 1
32         # 计算平均秩次
33         avg_rank = (i + j - 1) / 2.0
34         for k in range(i, j):
35             ranks[k] = avg_rank
36         i = j
37
38     # 计算正样本的秩次和
39     rank_sum = 0
40     pos_cnt = 0
41     for k in range(n):
42         if data[k][1] == 1: # 正样本
43             rank_sum += ranks[k]
44             pos_cnt += 1
45
46     neg_cnt = n - pos_cnt
47     auc = (rank_sum - pos_cnt * (pos_cnt - 1) / 2) / (pos_cnt * neg_cnt)
48     return auc
```